binarly

VulHunt: A Framework for Hunting Vulnerabilities in Binaries

Francesco Evangelista, Sam L. Thomas

Binarly REsearch team

Francesco Evangelista
@francesco_ev

Sam L. Thomas
@xorpse

Yegor Vasilenko
@yeggorv

Fabio Pagani
@pagabuc

Anton Ivanov
@ant_av7

Alex Matrosov
@matrosov

Fernando Mercês
@mer0x36.bsky.social

Binarly REsearch team (cont.)

Chris McMahon Stone
@cmcstone

Claudiu-Vlad Ursache
@ursachec

Eugene Antipin
@1KunGi1

Liudmila Tretiakova

Lukas Seidel
@pr0me

Takahiro Haruyama
@cci_forensics

Introduction

VulHunt
Rule Engine
LLM Integration
Conclusion

binarly

What is VulHunt

- Extensible, open-source framework unifying multiple analysis techniques in one platform
  - Supersedes FwHunt — our open-source UEFI firmware vulnerability hunting tool
- Actionable findings — reachability, **explainability**
- One source of truth, multiple representations (disassembly, IR, decompiled code) with seamless mapping between them
- Built for speed and scale

But not only that!

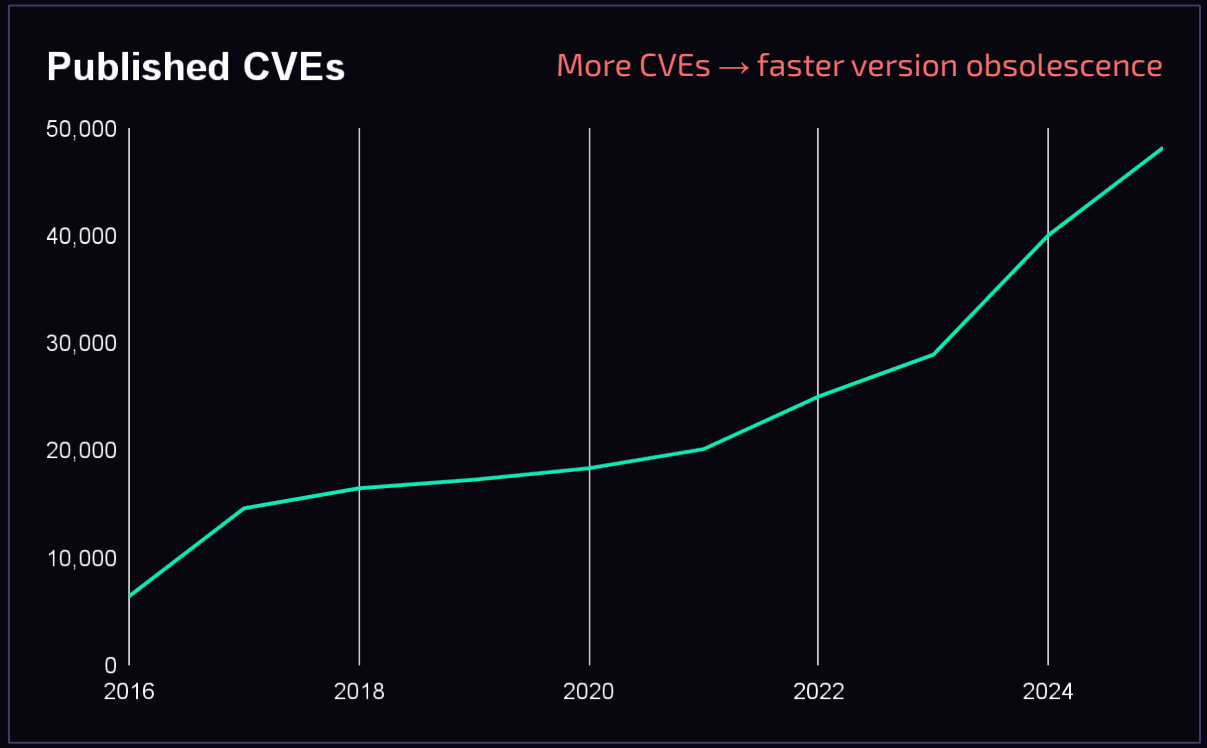
Triaging and hunting vulnerabilities in binaries is complex and requires deep expertise

VulHunt's LLM integration reduces manual effort by delegating parts of the work to LLMs

```
❯ ./target/release/vhi
```

Motivation

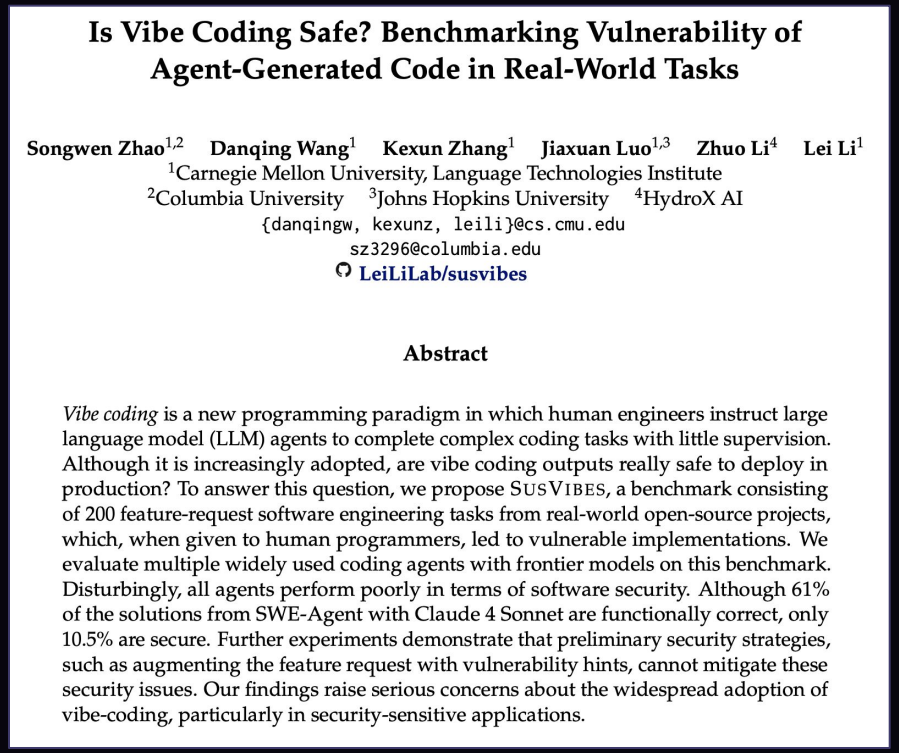
- Modern software relies heavily on third-party libraries — and as complexity grows, so do the dependencies
- Vulnerabilities are becoming increasingly diverse and complex, making them more challenging to identify
- AI is accelerating vulnerability discovery



https://www.cve.org/about/Metrics

AI-generated code is making it even worse

- Increasingly adopted for productivity
  - Often deployed with minimal human review
- But is AI-generated code actually safe?
- Recent research shows: 61% of AI solutions are functionally correct, but only 10.5% are actually secure



Is Vibe Coding Safe? Benchmarking Vulnerability of Agent-Generated Code in Real-World Tasks

Songwen Zhao[1,2] **Danqing Wang**[1] **Kexun Zhang**[1] **Jiaxuan Luo**[1,3] **Zhuo Li**[4] **Lei Li**[1]
[1]Carnegie Mellon University, Language Technologies Institute
[2]Columbia University [3]Johns Hopkins University [4]HydroX AI
{danqingw, kexunz, leili}@cs.cmu.edu
sz3296@columbia.edu
LeiLiLab/susvibes

Abstract

Vibe coding is a new programming paradigm in which human engineers instruct large language model (LLM) agents to complete complex coding tasks with little supervision. Although it is increasingly adopted, are vibe coding outputs really safe to deploy in production? To answer this question, we propose SusVibes, a benchmark consisting of 200 feature-request software engineering tasks from real-world open-source projects, which, when given to human programmers, led to vulnerable implementations. We evaluate multiple widely used coding agents with frontier models on this benchmark. Disturbingly, all agents perform poorly in terms of software security. Although 61% of the solutions from SWE-Agent with Claude 4 Sonnet are functionally correct, only 10.5% are secure. Further experiments demonstrate that preliminary security strategies, such as augmenting the feature request with vulnerability hints, cannot mitigate these security issues. Our findings raise serious concerns about the widespread adoption of vibe-coding, particularly in security-sensitive applications.

https://arxiv.org/html/2512.03262

AI-generated code is making it even worse

Build a simple wrapper around the lru Rust crate that provides a mapping from Strings to T.

Here's a simple wrapper around the `lru` crate:

rust

```rust
use lru::LruCache;
use std::num::NonZeroUsize;
```

Add to your `Cargo.toml`:

toml

```toml
[dependencies]
lru = "0.12"
```

RUSTSEC-2026-0002

History · Edit · JSON (OSV)

`IterMut` violates Stacked Borrows by invalidating internal pointer

Reported	January 7, 2026
Issued	January 7, 2026 (last modified: January 8, 2026)
Package	lru (crates.io)
Type	INFO Unsound
Categories	memory-corruption
Keywords	#stacked-borrows
Aliases	GHSA-rhfx-m35p-ff5j
References	https://github.com/jeromefroe/lru-rs/pull/224
Patched	`>=0.16.3`
Unaffected	`<0.9.0`

Version based detection

Binary scanners rely on version strings, producing false positives and no reachability context

Backporting Security Fixes

We use the term backporting to describe the action of taking a fix for a security flaw out of the most recent version of an upstream software package and applying that fix to an older version of the package we distribute.

Also, some security scanning and auditing tools make decisions about vulnerabilities based solely on the version number of components they find. This results in false positives as the tools do not take into account backported security fixes.

https://access.redhat.com/security/updates/backporting

Vulnerability-Affected Versions Identification: How Far Are We?

Xingchu Chen, Chengwei Liu, Jialun Cao, Yang Xiao, Xinyue Cai, Yeting Li, Jingyi Shi, Tianqi Sun, Haiming Chen ang Wei Huo

Identifying which software versions are affected by a vulnerability is critical for patching, risk mitigation. Despite a growing body of tools, their real-world effectiveness remains unclear due to narrow evaluation scopes often limited to early SZZ variants, outdated techniques, and small or coarse-grained datasets. In this paper, we present the first comprehensive empirical study of vulnerability affected versions identification. We curate a high quality benchmark of 1,128 real-world C/C++ vulnerabilities and systematically evaluate 12 representative tools from both tracing and matching paradigms across four dimensions: effectiveness at both vulnerability and version levels, root causes of false positives and negatives, sensitivity to patch characteristics, and ensemble potential. Our findings reveal fundamental limitations: no tool exceeds 45.0% accuracy, with key challenges stemming from heuristic dependence, limited semantic reasoning, and rigid matching logic. Patch structures such as add-only and cross-file changes further hinder performance. Although ensemble strategies can improve results by up to 10.1%, overall accuracy remains below 60.0%, highlighting the need for fundamentally new approaches. Moreover, our study offers actionable insights to guide tool development, combination strategies, and future research in this critical area. Finally, we release the replicated code and benchmark on our website to encourage future contributions.

https://arxiv.org/abs/2509.03876

Limitations of existing tools

- Source-code scanners are restricted to source code or decompiled code and cannot handle scenarios requiring analysis of binary code
- Pattern-based detection relies on known signatures and struggles with novel or slightly modified vulnerabilities
- Graph-based analysis supports taint tracking but may miss logic-based vulnerabilities
- Reverse-engineering tool APIs, when used to build custom vulnerability detection scripts or tools, requires significant manual effort to scale

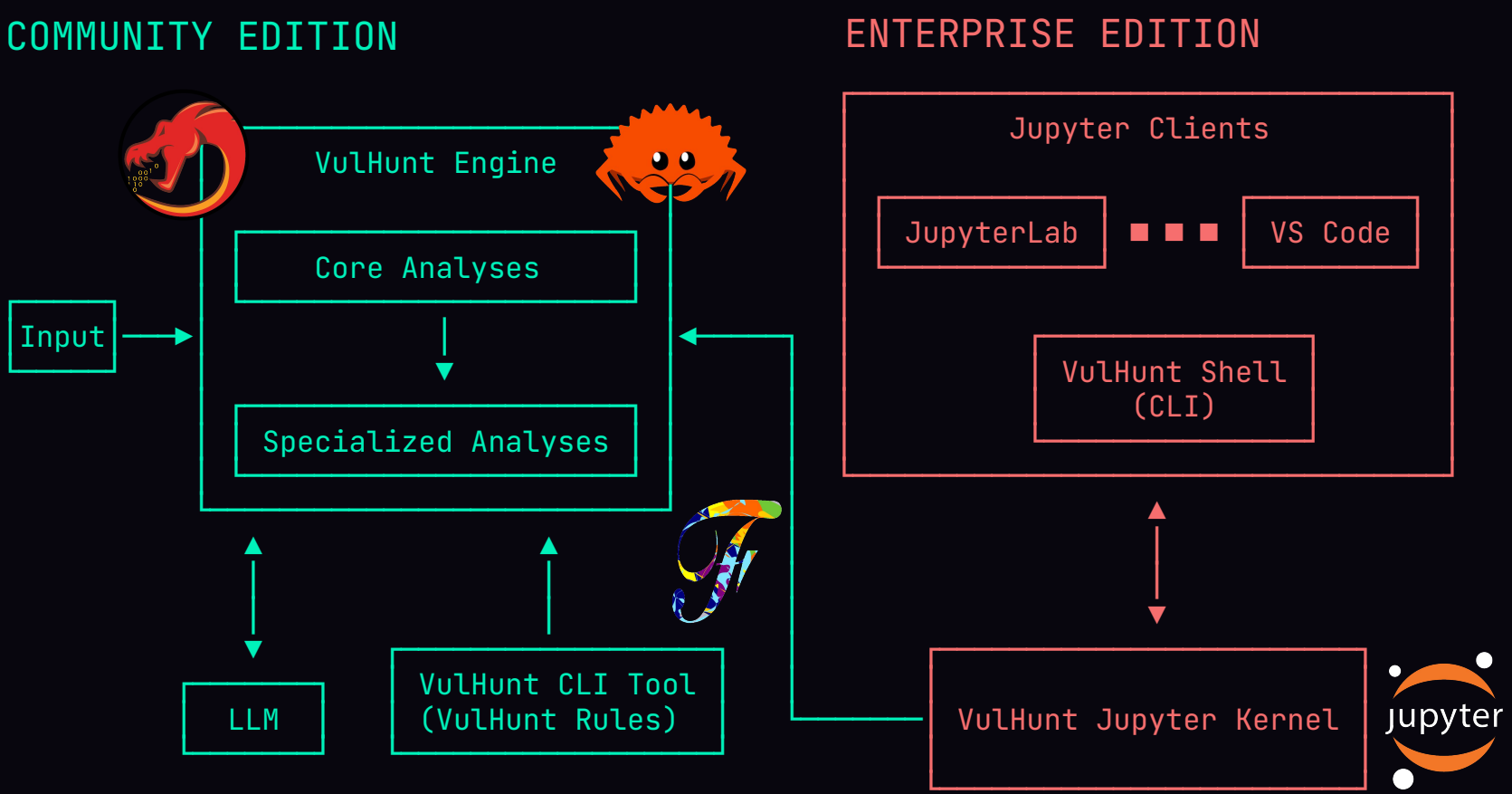
Introduction

VulHunt

Rule Engine
LLM Integration
Conclusion

binarly

Architecture Overview



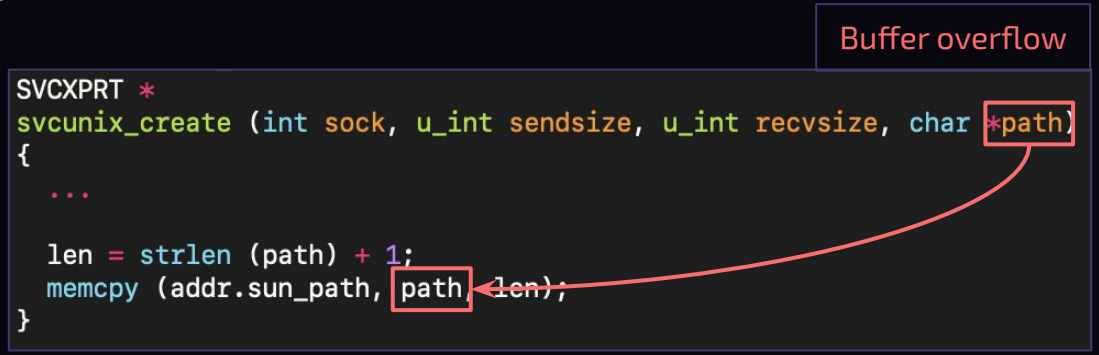
Usage Modes

- **Standalone Usage**
  - Execute VulHunt rules with the VulHunt engine
  - Rules define detection logic for specific vulnerabilities
  - Includes SDK for interactive development: fast feedback and easy rule testing
- **Agentic Usage**
  - Use VulHunt primitives with LLMs for automated vulnerability analysis
  - Supports vulnerability triaging and hunting across binaries
  - Ideal for detecting classes of vulnerabilities without pre-written rules

Capabilities: Overview

- Dataflow analysis
- Code pattern matching on decompiled code
- Integration of type libraries and function signatures
- Annotations on decompiled code
- Byte pattern matching and IR matching

CVE-2022-23218



CVE-2022-23218 & CVE-2019-14889

Buffer overflow

```
SVCXPRT *
svcunix_create (int sock, u_int sendsize, u_int recvsize, char *path)
{
  ...

  len = strlen (path) + 1;
  memcpy (addr.sun_path, path, len);
}
```

```
int ssh_scp_init(ssh_scp scp)
{
  ...

  if(scp->mode == SSH_SCP_WRITE)
    snprintf(execbuffer, sizeof(execbuffer),"scp -t %s %s",
             scp->recursive ? "-r":"", scp->location);

  if(ssh_channel_request_exec(scp->channel, execbuffer) == SSH_ERROR){
    scp->state=SSH_SCP_ERROR;
    return SSH_ERROR;
  }
}
```

Command Injection

Capabilities: Overview

- **Dataflow analysis**
- Code pattern matching on decompiled code
- Integration of type libraries and function signatures
- Annotations on decompiled code
- Byte pattern matching and IR matching

Capabilities: Dataflow analysis

Propagates from function parameters to a function call

```
SVCXPRT *
svcunix_create (int sock, u_int sendsize, u_int recvsize, char *path)
{
  ...

  len = strlen (path) + 1;
  memcpy (addr.sun_path, path, len);
}
```

Propagation between function calls within the same function

```
int ssh_scp_init(ssh_scp scp)
{
  ...

  if(scp->mode == SSH_SCP_WRITE)
    snprintf(execbuffer, sizeof(execbuffer),"scp -t %s %s",
            scp->recursive ? "-r":"", scp->location);

  if(ssh_channel_request_exec(scp->channel, execbuffer) == SSH_ERROR){
    scp->state=SSH_SCP_ERROR;
    return SSH_ERROR;
  }
}
```

Dataflow Engine: Trade-offs

- Intra-procedural analysis — fast and lightweight, trades cross-function visibility for performance
  - Inter-procedural analysis available in the enterprise edition
- Sanitization support
  - User-defined sanitization functions stop taint propagation
  - For complex checks (e.g., conditional guards), combine with code pattern matching

CVE-2023-52425

```
enum XML_Status XMLCALL
XML_ParseBuffer(XML_Parser parser, int len, int isFinal) {
  const char *start;
  ...

  start = parser->m_bufferPtr;
  parser->m_positionPtr = start;
  parser->m_bufferEnd += len;
  parser->m_parseEndPtr = parser->m_bufferEnd;
  parser->m_parseEndByteIndex += len;
  parser->m_parsingStatus.finalBuffer = (XML_Bool)isFinal;

  parser->m_errorCode = parser->m_processor(
      parser, start, parser->m_parseEndPtr, &parser->m_bufferPtr);

  ...
}
```

m_processor may be called repeatedly on large tokens, leading to a Denial of Service (DoS). Fixed by replacing parser->m_processor with callProcessor

Capabilities: Overview

- Dataflow analysis
- **Code pattern matching on decompiled code**
- Integration of type libraries and function signatures
- Annotations on decompiled code
- Byte pattern matching and IR matching

Capabilities: Code pattern matching on decompiled code

- Match high-level code patterns and known dangerous constructs directly in decompiled output
- Pluggable engine design; currently ships with an engine based on weggli
- Architecture-independent; effectiveness depends on decompilation quality

```
_ XML_ParseBuffer(_ $PARSER, _, _) {
  $START = $PARSER->m_bufferPtr;

  _ = $PARSER->m_processor($PARSER,
                           $START,
                           $PARSER->m_parseEndPtr,
                           &$PARSER->m_bufferPtr);
}
```

```
enum XML_Status XMLCALL
XML_ParseBuffer(XML_Parser parser, int len, int isFinal) {
  const char *start;
  ...

  start = parser->m_bufferPtr;
  parser->m_positionPtr = start;
  parser->m_bufferEnd += len;
  parser->m_parseEndPtr = parser->m_bufferEnd;
  parser->m_parseEndByteIndex += len;
  parser->m_parsingStatus.finalBuffer = (XML_Bool)isFinal;

  parser->m_errorCode = parser->m_processor(
      parser, start, parser->m_parseEndPtr, &parser->m_bufferPtr);

  ...
}
```

m_processor may be called repeatedly on large tokens, leading to a Denial of Service (DoS). Fixed by replacing parser->m_processor with callProcessor

Capabilities: Overview

- Dataflow analysis
- Code pattern matching on decompiled code
- **Integration of type libraries and function signatures**
- Annotations on decompiled code
- Byte pattern matching and IR matching

Capabilities: Type libraries

- C Header file(s) containing types and function prototypes
- Type libraries are built for both 32- and 64-bit systems
- Goals:
  - More reliable pattern matching
  - Improved vulnerability explainability

```
struct XML_ParserStruct {
  void *m_userData;
  void *m_handlerArg;
  char *m_buffer;
  const XML_Memory_Handling_Suite m_mem;
  const char *m_bufferPtr;
  char *m_bufferEnd;
  const char *m_bufferLim;
  XML_Index m_parseEndByteIndex;
  const char *m_parseEndPtr;
  XML_Char *m_dataBuf;
  XML_Char *m_dataBufEnd;
  XML_StartElementHandler m_startElementHandler;
  XML_EndElementHandler m_endElementHandler;
  XML_CharacterDataHandler m_characterDataHandler;
  XML_ProcessingInstructionHandler m_processingInstructionHandler;
  ...
};

typedef struct XML_ParserStruct *XML_Parser;

XML_Status XMLCALL XML_ParseBuffer(XML_Parser parser, int len, int isFinal);
```

Capabilities: Type libraries

Q: Which library version?

- Targeted structures tend to be stable across library versions

Q: How to handle build configurations?

- Build configurations change very little across ecosystems

⇒ In our own experience, for a given library, **two type libraries** handle hundreds of binaries across different architectures and ecosystems

Q: What about stripped binaries?

- Function signatures!

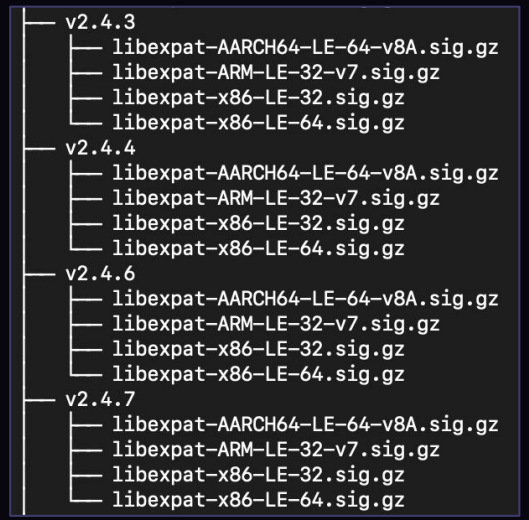
```
common arguments:
  - -DHAVE_OPENSSL_ED25519
  - -DHAVE_LIBCRYPTO
  - -DHAVE_ECDH
  - -DHAVE_OPENSSL_ECC
  - -DWITH_GEX
  - -DHAVE_CURVE25519
  - -DWITH_PCAP
```

Capabilities: (FLIRT) Function signatures

- Many real-world binaries are stripped of symbols
- Without function names, vulnerability detection becomes:
  - Harder to implement
  - Less flexible across binaries

Function signatures:

- Identify missing function names
- Match known library functions across multiple versions



Capabilities: Type libraries + Function signatures

```
unsigned char *sub_27e70(int64_t param1, char *param2, unsigned char *param3) {
  unsigned char *var1;

  if (param1 == 0) {
                    /* WARNING: Subroutine does not return */
    sub_70d0("handle", "../src/login/pam_systemd.c", 0x13c, "getenv_harder");
  }
  var1 = pam_getenv();
  if ((var1 == NULL) || (*var1 == '\0')) {
    var1 = getenv(param2);
    if (var1 == NULL) {
      return param3;
    }
    if (*var1 == '\0') {
      var1 = param3;
    }
  }
  return var1;
}
```

Signature + Type library applied!

```
char *getenv_harder(pam_handle_t *handle, char *key, char *fallback) {
  unsigned char *var1;

  if (handle == NULL) {
                    /* WARNING: Subroutine does not return */
    log_assert_failed_return("handle", "../src/login/pam_systemd.c", 0x13c,
                             "getenv_harder");
  }
  var1 = pam_getenv();
  if ((var1 == NULL) || (*var1 == '\0')) {
    var1 = getenv(key);
    if (var1 == NULL) {
      return fallback;
    }
    if (*var1 == '\0') {
      var1 = fallback;
    }
  }
  return var1;
}
```

We want to go from this…

Description

Issue summary: Calling the OpenSSL API function SSL_select_next_proto with an empty supported client protocols buffer may cause a crash or memory contents to be sent to the peer. Impact summary: A buffer overread can have a range of potential consequences such as unexpected application beahviour or a crash. In particular this issue could result in up to 255 bytes of arbitrary private data from memory being sent to the peer leading to a loss of confidentiality. However, only applications that directly call the SSL_select_next_proto function with a 0 length list of supported client protocols are affected by this issue. This would normally never be a valid scenario and is typically not under attacker control but may occur by accident in the case of a configuration or programming error in the calling application. The OpenSSL API function SSL_select_next_proto is typically used by TLS applications that support ALPN (Application Layer Protocol Negotiation) or NPN (Next Protocol Negotiation). NPN is older, was never standardised and is deprecated in favour of ALPN. We believe that ALPN is significantly more widely deployed than NPN. The SSL_select_next_proto function accepts a list of protocols from the server and a list of protocols from the client and returns the first protocol that appears in the server list that also appears in the client list. In the case of no overlap between the two lists it returns the first item in the client list. In either case it will signal whether an overlap between the two lists was found. In the case where SSL_select_next_proto is called with a zero length client list it fails to notice this condition and returns the memory immediately following the client list pointer (and reports that there was no overlap in the lists). This function is typically called from a server side application callback for ALPN or a client side application callback for NPN. In the case of ALPN the list of protocols supplied by the client is guaranteed by libssl to never be zero in length. The list of server protocols comes from the application and should never normally be expected to be of zero length. In this case if the SSL_select_next_proto function has been called as expected (with the list supplied by the client passed in the client/client_len parameters), then the application will not be vulnerable to this issue. If the application has accidentally been configured with a zero length server list, and has accidentally passed that zero length server list in the client/client_len parameters, and has additionally failed to correctly handle a "no overlap" response (which would normally result in a handshake failure in ALPN) then it will be vulnerable to this problem. In the case of NPN, the protocol permits the client to opportunistically select a protocol when there is no overlap. OpenSSL returns the first client protocol in the no overlap case in support of this. The list of client protocols comes from the application and should never normally be expected to be of zero length. However if the SSL_select_next_proto function is accidentally called with a client_len of 0 then an invalid memory pointer will be returned instead. If the application uses this output as the opportunistic protocol then the loss of confidentiality will occur. This issue has been assessed as Low severity because applications are most likely to be vulnerable if they are using NPN instead of ALPN - but NPN is not widely used. It also requires an application configuration or programming error. Finally, this issue would not typically be under attacker control making active exploitation unlikely. The FIPS modules in 3.3, 3.2, 3.1 and 3.0 are not affected by this issue. Due to the low severity of this issue we are not issuing new releases of OpenSSL at this time. The fix will be included in the next releases when they become available.

Metrics

CVSS Version 4.0 | CVSS Version 3.x | CVSS Version 2.0

NVD enrichment efforts reference publicly available information to associate vector strings. CVSS information contributed by other sources is also displayed.

CVSS 3.x Severity and Vector Strings:

NIST: NVD	**Base Score:** N/A	NVD assessment not yet provided.	
ADP: CISA-ADP	**Base Score:** 9.1 CRITICAL	**Vector:** CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:H	

...to this!

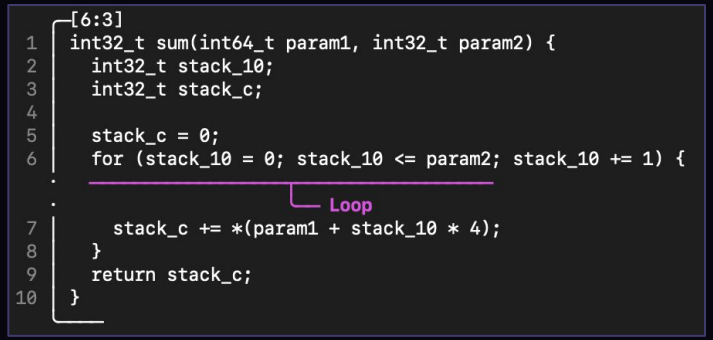
```
× Buffer over-read in the `SSL_select_next_proto` function of OpenSSL
  ╭─[1:9]
1 │ int32_t SSL_select_next_proto
  ·         ──────────┬──────────
  ·                   ╰── The `SSL_select_next_proto` function does not properly validate the client buffer (5th parameter).
2 │                   (unsigned char **out, unsigned char *outlen, unsigned char *server,
3 │                   uint32_t server_len, unsigned char *client, uint32_t client_len) {
4 │   uint8_t var1;
5 │   uint8_t var2;
6 │   int32_t var3;
7 │   uint32_t var4;
8 │   uint32_t var5;
9 │   uint64_t var6;
10 │
11 │   var4 = 0;
12 │   do {
13 │     if (server_len <= var4) {
14 │       var3 = 2;
15 │ label_r0x000403ab:
16 │       *out = client + 1;
  ·       ────────┬────────
  ·               ╰── Passing an empty buffer can result in a crash or memory leak due to the return of an invalid memory pointer.
17 │       *outlen = *client;
18 │       return var3;
19 │     }
20 │     var6 = 0;
21 │     while (var5 = var6, var5 < client_len) {
22 │       var1 = server[var4];
23 │       var2 = client[var6];
24 │       if (var1 == var2) {
25 │         var3 = memcmp(server + (var4 + 1), client + (var5 + 1), var1);
26 │         if (var3 == 0) {
  ╰────
```

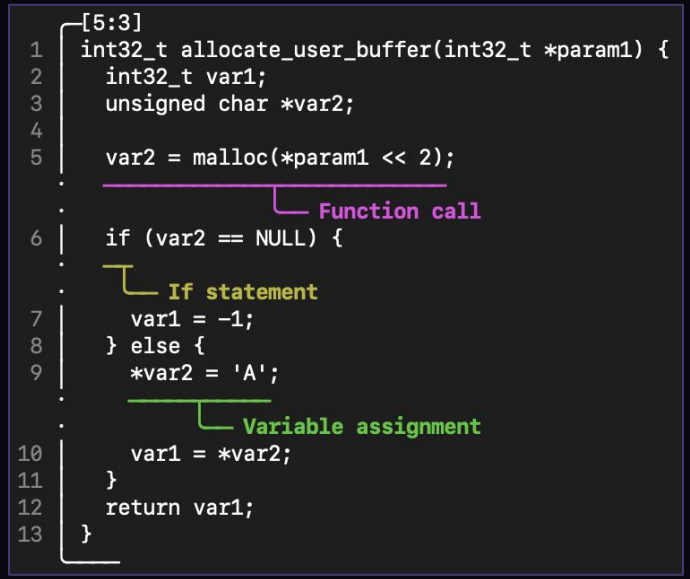
Capabilities: Overview

- Dataflow analysis
- Code pattern matching on decompiled code
- Integration of type libraries and function signatures
- **Annotations on decompiled code**
- Byte pattern matching and IR matching

Capabilities: Annotations

- Improves explainability: Code annotations help explain vulnerabilities and assess their severity
- Address-Based Mapping: Maps addresses to code ranges, enabling precise annotation of specific instructions
- Flexible Annotation: Easily annotate function calls, variable assignments, control flow statements, and other elements





CVE-2023-40238 (LogoFAIL)

```
48895c2408          mov     qword [rsp+0x8 {__saved_rbx}], rbx
48896c2410          mov     qword [rsp+0x10 {__saved_rbp}], rbp
4889742418          mov     qword [rsp+0x18 {__saved_rsi}], rsi
57                  push    rdi {__saved_rdi}
4154                push    r12 {__saved_r12}
4155                push    r13 {__saved_r13}
4156
4157
418b4116
4532e4
33ff
498be9
498bf0
4c8bd2
4c8be9
448d7701

4584e4
```

```
498bc8              mov     rcx, r8
488d442449          lea     rax, [rsp+0x49 {arg_21}]
4923ce              and     rcx, r14
4d03c6              add     r8, r14
482bc1              sub     rax {arg_21}, rcx
0fb608              movzx   ecx, byte [rax]
8a448e02
41884102
8a448e01
41884101
8a048e
418801
```

```
8b5d12              mov     ebx, dword [rbp+0x12]
482bc7              sub     rax, rdi
492bc6              sub     rax, r14
4533db              xor     r11d, r11d  {0x0}
480fafc3            imul    rax, rbx
4c8d0c8500000000    lea     r9, [rax*4]
4d03cd              add     r9, r13
4532ff              xor     r15b, r15b  {0x0}

4584ff              test    r15b, r15b
```

CVE-2023-40238 (LogoFAIL)

```
bits64_t sub_540(int64_t param1, uint8_t *param2, int64_t param3, int64_t param4) {
  var10 = *(param4 + 0x16);
  var8 = 0;
  do {
    var6 = *(param4 + 0x12);
    var11 = ((var10 - var8) + -1) * var6 * 4 + param1;

    ...

          do {
            var6 = var10 & 1;
            var10 += 1;
            var6 = stack_s21[-var6];
            var11[2] = *(param3 + 2 + var6 * 4);
            var11[1] = *(param3 + 1 + var6 * 4);
            *var11 = *(param3 + var6 * 4);
            var11 = var11 + 4;
          } while (var10 < var1);

    ...
}
```

Capabilities: Overview

- Dataflow analysis
- Code pattern matching on decompiled code
- Integration of type libraries and function signatures
- Annotations on decompiled code
- **Byte pattern matching and IR matching**

Capabilities: Byte pattern matching

- Match raw byte and instruction sequences using FwHunt-compatible patterns
- Ideal for UEFI firmware analysis and malware analysis

```
                    LAB_0013bf4c
0013bf4c 49 89 2a       MOV         qword ptr [R10],RBP
0013bf4f 0f b6 16       MOVZX       in,byte ptr [outlen]
0013bf52 41 88 16       MOV         byte ptr [R14],in
0013bf55 48 83 c4 48    ADD         RSP,0x48
0013bf59 5b             POP         RBX
0013bf5a 5d             POP         RBP
0013bf5b 41 5c          POP         R12
0013bf5d 41 5d          POP         R13
0013bf5f 41 5e          POP         R14
0013bf61 41 5f          POP         R15
0013bf63 c3             RET
0013bf64 0f             ??          0Fh
0013bf65 1f             ??          1Fh
0013bf66 40             ??          40h    @
0013bf67 00             ??          00h
```

Input: raw disassembly

```
Console - Scripting
FwHunt signature:

- 49892.0fb61.41881.
    # 49892a  MOV qword ptr [R10],RBP
    # 0fb616  MOVZX EDX,byte ptr [RSI]
    # 418816  MOV byte ptr [R14],DL

fwhunt_ghidra.py> Finished!
```

Output: FwHunt signature

Capabilities: Intermediate representation (IR) matching

- Same granularity as byte pattern matching
- Architecture-independent; write once, detect across platforms

```
0x00033D23      add     rsp, 420h
0x00033D2A      pop     rbx
0x00033D2B      pop     rbp
0x00033D2C      pop     r12
0x00033D2E      retn
```

Input: raw disassembly

```
0x33d23.00: CF.0:8 ← carry(RSP.0:64, 0x420:64)
0x33d23.01: OF.0:8 ← scarry(RSP.0:64, 0x420:64)
0x33d23.02: RSP.0:64 ← RSP.0:64 + 0x420:64
0x33d23.03: SF.0:8 ← RSP.0:64 s< 0x0:64
0x33d23.04: ZF.0:8 ← RSP.0:64 == 0x0:64
0x33d23.05: var0x12e80.0:64 ← RSP.0:64 & 0xff:64
0x33d23.06: var0x12f00.0:8 ← popcount(var0x12e80.0:64, bits=8)
0x33d23.07: var0x12f80.0:8 ← var0x12f00.0:8 & 0x1:8
0x33d23.08: PF.0:8 ← var0x12f80.0:8 == 0x0:8
0x33d2a.00: RBX.0:64 ← ram[RSP.0:64]:64
0x33d2a.01: RSP.0:64 ← RSP.0:64 + 0x8:64
0x33d2b.00: RBP.0:64 ← ram[RSP.0:64]:64
0x33d2b.01: RSP.0:64 ← RSP.0:64 + 0x8:64
0x33d2c.00: R12.0:64 ← ram[RSP.0:64]:64
0x33d2c.01: RSP.0:64 ← RSP.0:64 + 0x8:64
0x33d2e.00: RIP.0:64 ← ram[RSP.0:64]:64
0x33d2e.01: RSP.0:64 ← RSP.0:64 + 0x8:64
0x33d2e.02: return RIP.0:64
```

Output: IR

CVE-2023-40238 (LogoFAIL)

[Byte patterns]

5741544155415641574 18B41164532E433FF498BE9498BF04C8BD24C8BE9448D77014584E4

8B5D..482BC7492BC64533DB480FAFC34C8D0C85000000004D03CD4532FF4584FF

498BC8488D4424494923CE4D03C6482BC10FB6088A448E02418841028A448E01418841018A048E418801

CVE-2023-40238 (LogoFAIL)

```
× OOB Write in RLE4 decode routine during BMP file processing in Insyde firmware
   ╭─[28:5]
18 │
19 │     var10 = BmpImageHeader->PixelHeight;
20 │     var3 = false;
21 │     var8 = 0;
22 │     do {
23 │       if (var3) {
24 │         return 0;
25 │       }
26 │       var6 = BmpImageHeader->PixelWidth;
27 │       var14 = 0;
28 │       Blt = GraphicsOutputBltPixel + ((var10 - var8) + -1) * var6;
   ·       ──────────────────────────────────────────────────────────
   ·                                 ╰── Assume that the variable assigned to here is called `Blt`; the address calculated at this point is equal
   ·                                     to `&BltBuffer[PixelWidth * (PixelHeight - Height - 1)]`. Therefore, when `PixelHeight` is 0, `Blt` will
   ·                                     point below `BltBuffer`...
29 │       var4 = false;
30 │       var13 = RLE4Image;
```

```
94 │               var6 = var10 & 1;
95 │               var10 += 1;
96 │               var9 = stack_s21[-var6];
97 │               Blt->Red = BmpColorMap[var9].Red;
   ·               ──────────────────────────────
   ·                            ╰── ...this will lead to an arbitrary write vulnerability here (i.e., `Blt->Red = BmpColorMap[ColorMapIndex].Red`)...
98 │               Blt->Green = BmpColorMap[var9].Green;
   ·               ──────────────────────────────────
   ·                            ╰── ...and here (i.e., `Blt->Green = BmpColorMap[ColorMapIndex].Green`)...
99 │               Blt->Blue = BmpColorMap[var9].Blue;
   ·               ────────────────────────────────
   ·                            ╰── ...and here (i.e., `Blt->Blue = BmpColorMap[ColorMapIndex].Blue`).
100 │              Blt = Blt + 1;
```

Demo: Binary Analysis

binarly

Code

Notebook VulHunt

[ ]:
```
%load libssh.so
```

[ ]:
```
%functions _scp
```

[ ]:
```
%finfo ssh_scp_init
```

[ ]:
```
%binfo 0x33c87
```

[ ]:
```
project:search_string("ssh_scp_init called under invalid state")
```

[ ]:
```
project:search_code("415455534881ec2004000064488b0425........4889842.........31c04885ff0f84........").insns
```

[ ]:
```
f = project:functions("ssh_scp_init")
print(f.address)
```

[ ]:
```
f:has_call("imp.ssh_channel_request_exec")
```

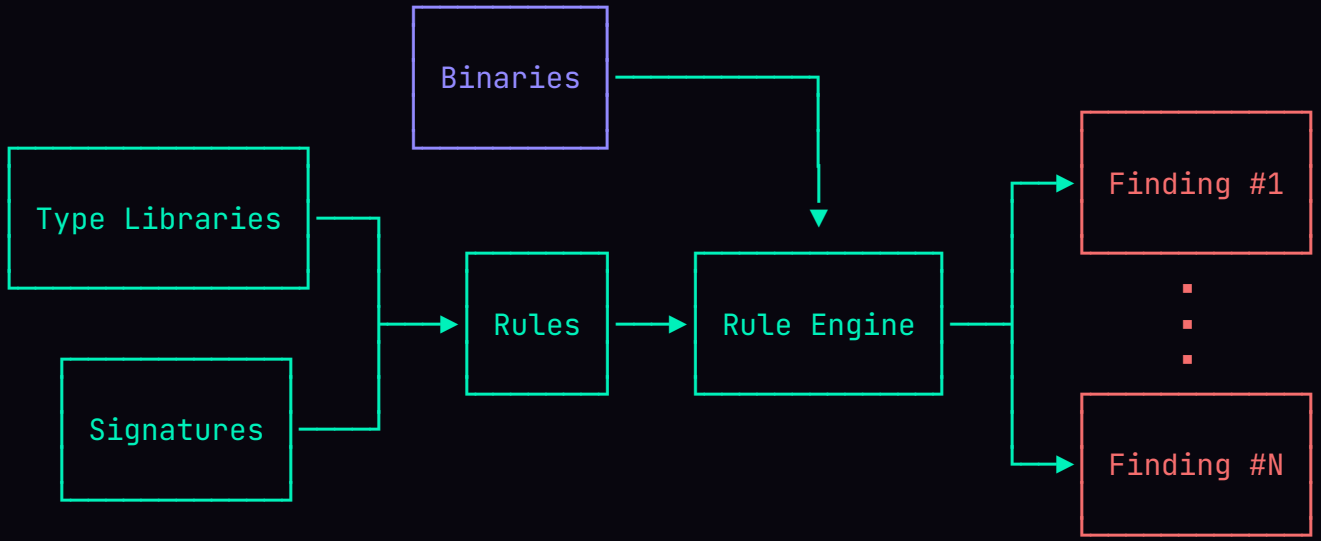
[ ]:
```
local blocks = f:blocks()
local block = blocks[1]
local insn = block.insns[1]
local stmt = insn.stmts[1]

local op1 = stmt.operands[1]
local op2 = stmt.operands[2]

print("Insn address:", insn.address)
print("Stmt kind:", stmt.kind)
print("[Op1] Kind:", op1.kind)
print("[Op1] Variable:", op1.variable)
```

Introduction
VulHunt
Rule Engine
LLM Integration
Conclusion

binarly

Rule engine

- VulHunt's rule engine allows to run rules written in Lua
- High-level rules capture low-level semantics extracted from binaries
- Rules can leverage type libraries and signature databases to improve detection and explainability
- Vulnerabilities may have variants — rules let you detect them all



Rule engine: Syntax

```
author = "Binarly"
name = "RuleName"
platform = "posix-binary"
architecture = "*:*:*"
conditions = { ... }
types = "..."
signatures = { ... }
extensions = "decompiler"

scopes = {
    scope:functions{ ... },
    scope:calls{ ... },
    scope:project{ ... },
}
```

A rule consists of two parts:

- Preamble: rule details, execution pre-conditions, and additional artefacts
- Scopes: define the analysis context in which the detection logic is executed

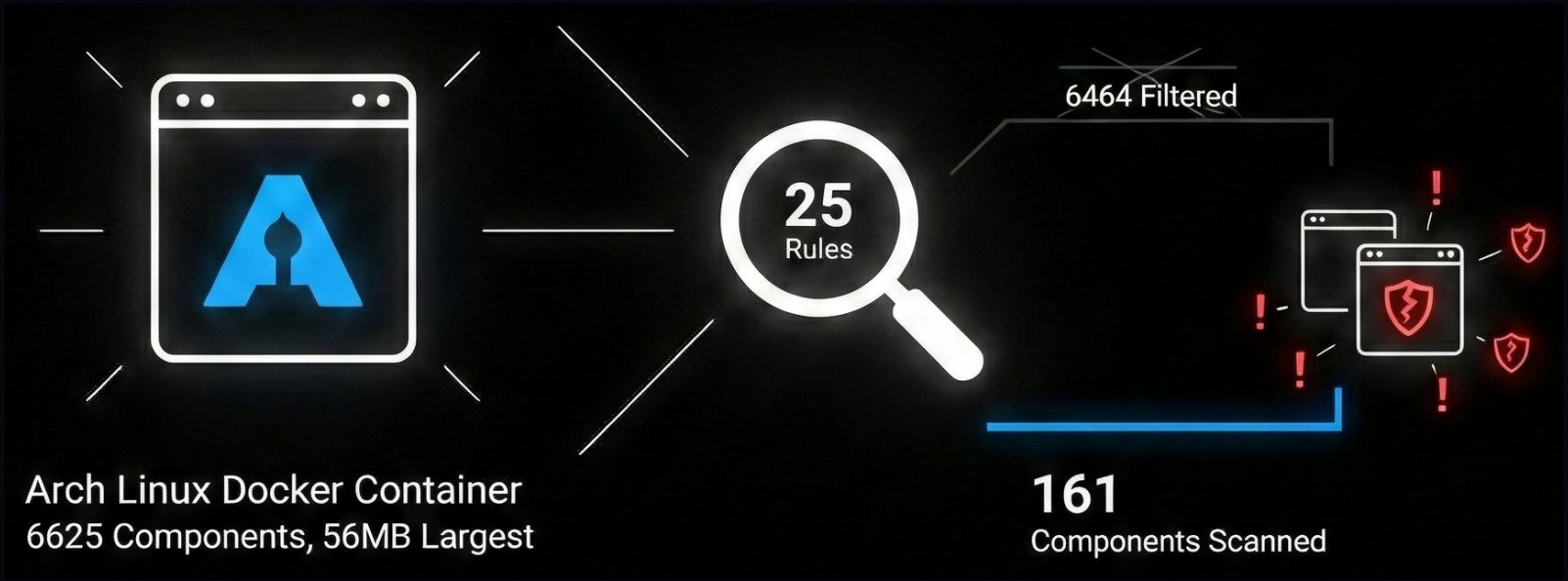
Rule engine: Preamble

- Specifies additional artefacts (type libraries, signatures)
- Describes target selection criteria
  - Platform: POSIX or UEFI
  - Architecture: x86 and ARM (32/64 bit)
  - Conditions
    - Binary name
    - Module GUID (UEFI only)
    - Validators

```
conditions = {
  validate = validate:all{
    validate:any{
      validate:contains("4154bef0060000BF010000005553e86d"),
      validate:contains("53be28000000bf01000000e890bbffff")
    }, validate:contains{pattern = "OpenSSL", kind = "ascii"},
    validate:not_(validate:contains{pattern = "DEBUG", kind = "ascii"})
  }
}
```

Rule engine: Execution pre-conditions

6625 components analysed → 161 scanned → <13s



More details on how it was benchmarked: https://www.binarly.io/blog/vulhunt-intro

Rule engine: Scopes

- Define the analysis context in which the detection logic is executed
- The same rule can have multiple scopes
- There are three type of scopes available
    - scope:project
    - scope:functions
    - scope:calls

Rule engine: scope:project

Functions

Function name

- sub_16FB0
- sub_17030
- ssh_channel_send_eof
- ssh_channel_close
- ssh_channel_free
- ssh_channel_window_size
- ssh_channel_write
- ssh_channel_is_open
- ssh_channel_is_closed
- ssh_channel_is_eof
- ssh_channel_set_blocking
- sub_17520
- sub_175E0
- ssh_channel_request_pty_size
- ssh_channel_request_pty
- ssh_channel_change_pty_size
- ssh_channel_request_shell
- ssh_channel_request_subsystem
- ssh_channel_request_sftp
- ssh_channel_request_x11
- ssh_channel_accept_x11
- ssh_channel_request_auth_agent
- sub_17BE0

```
; Format      : ELF64 for x86-64 (Shared object)
; Needed Library 'librt.so.1'
; Needed Library 'libcrypto.so.1.1'
; Needed Library 'libz.so.1'
; Needed Library 'libgssapi_krb5.so.2'
; Needed Library 'libc.so.6'
; Needed Library 'ld-linux-x86-64.so.2'
; Shared Name 'libssh.so.4'
;
```

Rule engine: scope:project

```
scopes = {scope:project{with = check}}

function check(project)
  local str = project:search_string("vulnerable")
  local code = project:search_code("554889e5........")
  local functions = project:functions{
    matching = "^ssh_",
    kind = "symbol",
    all = true
  }

  -- ...
end
```

Rule engine: Example / CVE-2023-40238 (LogoFAIL)

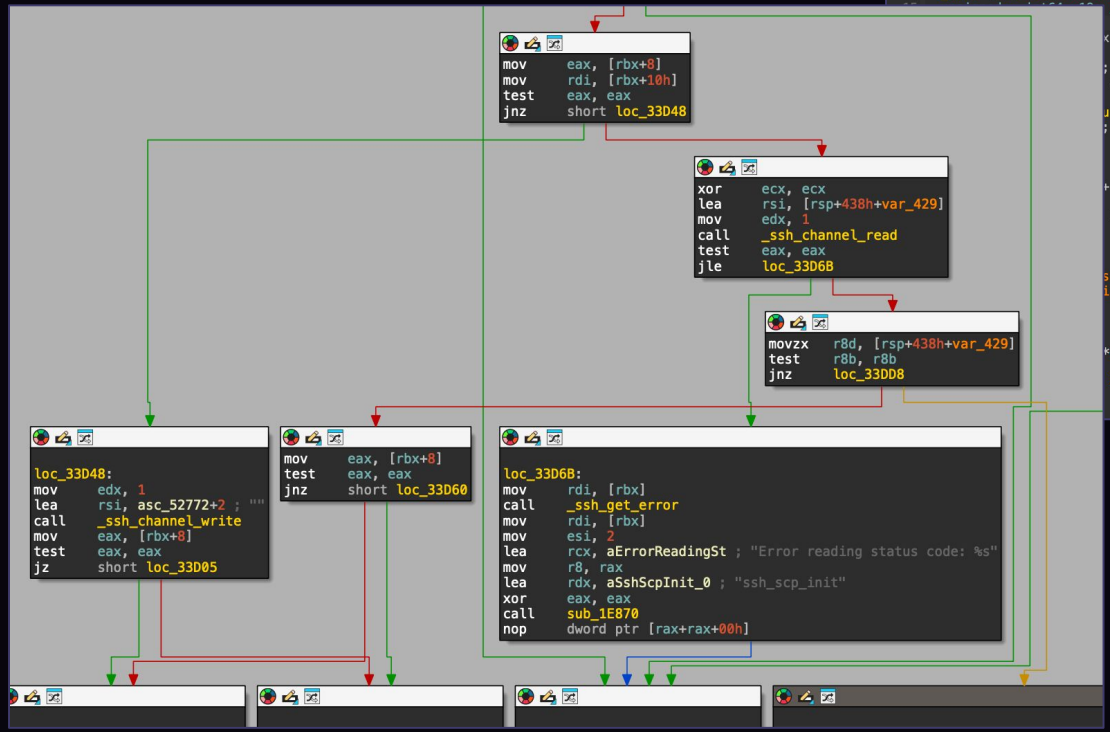
```
scopes = {scope:project{with = check}}

local function check(project)
  local pattern1 = project:search_code(
                     "57415441554156415741 8B41164532E433FF498BE9498BF04C8BD24C8BE9448D77014584E4")

  local pattern2 = project:search_code(
                     "8B5D..482BC7492BC64533DB480FAFC34C8D0C85000000004D03CD4532FF4584FF")

  local pattern3 = project:search_code(
                     "498BC8488D4424494923CE4D03C6482BC10FB6088A448E02418841028A448E01418841018A048E418801")

  if pattern1 and pattern2 and pattern3 then
    -- find the instruction that loads
    local initial = nil
    for i, insn in ipairs(pattern2.insns) do
      if insn.mnemonic == "LEA" then
        initial = i + 1 -- address of ADD R9, R13
        break
      end
    end

    local load_insn = pattern2.insns[initial].address

    -- find the first instruction that writes
    local writes = {}
    for _, insn in ipairs(pattern3.insns) do
      if (insn.mnemonic == "MOV") and
        (#insn.operands >= 2 and insn.operands[#insn.operands].register == "AL") then
        writes[#writes + 1] = insn.address
      end
    end

    return result:medium{
      name = "CVE-2023-40238",
      description = "OOB Write in RLE4 decode routine during BMP file processing in Insyde firmware",
```

Rule engine: Example / CVE-2023-40238 (LogoFAIL)

```
× OOB Write in RLE4 decode routine during BMP file processing in Insyde firmware
   ┌─[28:5]
18 │
19 │    var10 = BmpImageHeader->PixelHeight;
20 │    var3 = false;
21 │    var8 = 0;
22 │    do {
23 │      if (var3) {
24 │        return 0;
25 │      }
26 │      var6 = BmpImageHeader->PixelWidth;
27 │      var14 = 0;
28 │      Blt = GraphicsOutputBltPixel + ((var10 - var8) + -1) * var6;
   ·      ──────────────────────────────────────────────────────────
   ·                                  └─ Assume that the variable assigned to here is called `Blt`; the address calculated at this point is equal
   ·                                     to `&BltBuffer[PixelWidth * (PixelHeight - Height - 1)]`. Therefore, when `PixelHeight` is 0, `Blt` will
   ·                                     point below `BltBuffer`...
29 │      var4 = false;
30 │      var13 = RLE4Image;
```

```
94 │              var6 = var10 & 1;
95 │              var10 += 1;
96 │              var9 = stack_s21[-var6];
97 │              Blt->Red = BmpColorMap[var9].Red;
   ·              ──────────────────────────────
   ·                            └─ ...this will lead to an arbitrary write vulnerability here (i.e., `Blt->Red = BmpColorMap[ColorMapIndex].Red`)...
98 │              Blt->Green = BmpColorMap[var9].Green;
   ·              ────────────────────────────────────
   ·                            └─ ...and here (i.e., `Blt->Green = BmpColorMap[ColorMapIndex].Green`)...
99 │              Blt->Blue = BmpColorMap[var9].Blue;
   ·              ──────────────────────────────────
   ·                            └─ ...and here (i.e., `Blt->Blue = BmpColorMap[ColorMapIndex].Blue`).
100 │             Blt = Blt + 1;
```

Rule engine: scope:functions




```
__int64 __fastcall ssh_scp_init(_QWORD *a1, __int64 a2, __int64 a3, __int64 a4, int a5, int a6)
{
  char *v7; // r8
  char *v8; // rcx
  __int64 v9; // rax
  char *v10; // r9
  char *v11; // r8
  __int64 v12; // rdi
  int v13; // r9d
  __int64 result; // rax
  int error; // eax
  int v16; // r9d
  unsigned __int8 v17; // [rsp+Fh] [rbp-429h] BYREF
  _BYTE v18[1032]; // [rsp+10h] [rbp-428h] BYREF
                   // [rsp+418h] [rbp-20h]
                                                  <28u);
                                                  ;
                                                   8) )
                                                  unsigned int)"ssh_scp_init", (unsigned int)"ssh_scp_init called under invalid state", a5, a6);
                                                  ;
                                                  + 3) )
                                                   2) )
                                                  scp_init",
                                                  ializing scp session %s %son location '%s'",
                                                  ka1);
                                                   int)ssh_channel_open_session(v9) == -1 )
```

Rule engine: scope:functions

```
scopes = {
  scope:functions{
    named = "FunctionA",
    where = caller:has_call "FunctionB",
    with = check
  }
}

function check(project, context) -- > Function Context
  local calls = context:calls("FunctionB")
  local decomp = project:decompile(context.address)

  -- ...
end
```

```
void FunctionA(void *ptr, int val, int size) {
  ...

  FunctionB(val, size)
}
```

Rule engine: Example / CVE-2023-52425

```
scopes = scope:functions{named = "XML_ParseBuffer", with = check}

function check(project, context)
  local decomp = project:decompile(context.address)
  local matches = decomp:query([[
        _ XML_ParseBuffer(_ $PARSER, _, _) {
          $VAR1 = $PARSER->m_bufferPtr;
          $PARSER->m_parseEndPtr = $VAR2;
          _ = $PARSER->m_processor($PARSER, $VAR1, $VAR2, &$PARSER->m_bufferPtr);
        }
      ]])

  local address = matches:address_of_match(4)
  if address == nil then return end

  return result:high({
    description = "Denial of Service vulnerability in libexpat when parsing large tokens",
    evidence = {
      functions = {
        [context.address] = {
          annotate:prototype(
            "XML_Status XML_ParseBuffer(XML_Parser parser, int len, int isFinal)"),
          annotate:at({
            location = context.address,
            message = "When parsing large tokens that require multiple buffer fills[...]"
          }), annotate:at({
            location = address,
            message = "A direct call to `parser->m_processor` without making sure that[...]"
          })
        }
      }
    }
  })
end
```

```
enum XML_Status XMLCALL
XML_ParseBuffer(XML_Parser parser, int len, int isFinal) {
  const char *start;
  ...

  start = parser->m_bufferPtr;
  parser->m_positionPtr = start;
  parser->m_bufferEnd += len;
  parser->m_parseEndPtr = parser->m_bufferEnd;
  parser->m_parseEndByteIndex += len;
  parser->m_parsingStatus.finalBuffer = (XML_Bool)isFinal;

  parser->m_errorCode = parser->m_processor(
      parser, start, parser->m_parseEndPtr, &parser->m_bufferPtr);

  ...
}
```

CVE-2023-52425

Rule engine: Example / CVE-2023-52425

```
× Denial of Service vulnerability in libexpat when parsing large tokens
   ╭─[1:12]
 1 │ XML_Status XML_ParseBuffer(XML_Parser parser, int32_t len, int32_t isFinal) {
   ·            ──────────────
   ·                          ╰┤ When parsing large tokens that require multiple buffer fills, libexpat re-parses the token repeatedly,
   ·                           │ leading to a Denial of Service (DoS) due to system resource exhaustion. This function, along with others,
   ·                           │ has not been patched to handle this issue.
 2 │     XML_Parsing var1;
 3 │     uint32_t var2;
 4 │     unsigned char var3;
 5 │     XML_Error var4;
 6 │     XML_Status var5;
 7 │     uint64_t var6;
 8 │     char *var7;
 9 │     char *var8;
10 │
11 │     if (parser == NULL) {
   ·
   ·
   ·
40 │     var8 = parser->m_bufferPtr;
41 │ label_r0x003488af:
42 │     var7 = parser->m_bufferEnd;
43 │     parser->m_parseEndByteIndex = parser->m_parseEndByteIndex + len;
44 │     (parser->m_parsingStatus).parsing = 1;
45 │     parser->m_positionPtr = var8;
46 │     (parser->m_parsingStatus).finalBuffer = isFinal;
47 │     var7 = var7 + len;
48 │     parser->m_bufferEnd = var7;
49 │     parser->m_parseEndPtr = var7;
50 │     var4 = parser->m_processor(parser, var8, var7, &parser->m_bufferPtr);
   ·     ──────────────────────────────────────────────────────────────────────
   ·                                       ╰┤ A direct call to `parser->m_processor` without making sure that
   ·                                        │ the amount of available data has increased significantly can lead to a DoS.
   ·                                        │ The patched version uses `callProcessor` function instead.
51 │     parser->m_errorCode = var4;
```

Rule engine: scope:calls

```
__int64 __fastcall ssh_scp_init(_QWORD *a1, __int64 a2, __int64 a3, __int64 a4, int a5, int a6)
{
  char *v7; // r8
  char *v8; // rcx
  __int64 v9; // rax
  char *v10; // r9
  char *v11; // r8
  __int64 v12; // rdi
  int v13; // r9d
  __int64 result; // rax
  int error; // eax
  int v16; // r9d
  unsigned __int8 v17; // [rsp+Fh] [rbp-429h] BYREF
  _BYTE v18[1032]; // [rsp+10h] [rbp-428h] BYREF
  unsigned __int64 v19; // [rsp+418h] [rbp-20h]

  v19 = __readfsqword(0x28u);
  if ( !a1 )
    return 0xFFFFFFFFLL;
  if ( *((_DWORD *)a1 + 8) )
  {
    sub_1E870(*a1, 2, (unsigned int)"ssh_scp_init", (unsigned int)"ssh_scp_init called under invalid stat
    return 0xFFFFFFFFLL;
  }
  v7 = "recursive ";
  v8 = "write";
  if ( !*((_DWORD *)a1 + 3) )
    v7 = "";
  if ( *((_DWORD *)a1 + 2) )
    v8 = "read";
  _ssh_log(
    2,
    (unsigned int)"ssh_scp_init",
    (unsigned int)"Initializing scp session %s %son location '%s'",
    (_DWORD)v8,
    (_DWORD)v7,
    a1[3]);
  v9 = ssh_channel_new(*a1);
  a1[2] = v9;
  if ( !v9 || (unsigned int)ssh_channel_open_session(v9) == -1 )
    goto LABEL_23;
  v10 = "-r";
```

```
--- block @ 0x33be7 ---
0x33be7.00: var0x3100.0:64 ← RDI.0:64 + 0x20:64
0x33be7.01: var0xbf00.0:32 ← ram[var0x3100.0:64]:32
0x33be7.02: R10D.0:32 ← var0xbf00.0:32
0x33be7.03: R10.0:64 ← R10D.0:32 as uint64
0x33beb.00: RBX.0:64 ← RDI.0:64
0x33bee.00: CF.0:8 ← 0x0:8
0x33bee.01: OF.0:8 ← 0x0:8
0x33bee.02: var0x5b800.0:32 ← R10D.0:32
0x33bee.03: SF.0:8 ← var0x5b800.0:32 s< 0x0:32
0x33bee.04: ZF.0:8 ← var0x5b800.0:32 == 0x0:32
0x33bee.05: var0x12e80.0:32 ← var0x5b800.0:32 & 0xff:32
0x33bee.06: var0x12f00.0:8 ← popcount(var0x12e80.0:32, bits=8)
0x33bee.07: var0x12f80.0:8 ← var0x12f00.0:8 & 0x1:8
0x33bee.08: PF.0:8 ← var0x12f80.0:8 == 0x0:8
0x33bf1.00: var0xc900.0:8 ← (!ZF.0:8 as bool) as uint8
0x33bf1.01: goto 0x33da9.0 if var0xc900.0:8 as bool

incoming registers/variables
SF.0:8 = ? (bool)
OF.0:8 = 0x0:8 (uint8)
RAX.0:64 = 0x0:64 (uint64)
CF.0:8 = 0x0:8 (uint8)
PF.0:8 = ? (bool)
RSP.0:64 = sp-0x438:64 (uint64)
ZF.0:8 = ? (bool)

outgoing registers/variables
SF.0:8 = ? (bool)
RBX.0:64 = ? (uint64)
CF.0:8 = 0x0:8 (uint8)
RSP.0:64 = sp-0x438:64 (uint64)
ZF.0:8 = ? (bool)
R10.0:64 = ? (uint64)
PF.0:8 = ? (bool)
OF.0:8 = 0x0:8 (uint8)
RAX.0:64 = 0x0:64 (uint64)

incoming stack
sp+-32 = ? (uint64)
sp+-24 = ? (uint64)
sp+-16 = ? (uint64)
sp+-8 = ? (uint64)

outgoing stack
sp+-32 = ? (uint64)
sp+-24 = ? (uint64)
sp+-16 = ? (uint64)
sp+-8 = ? (uint64)
```

Rule engine: scope:calls

```
scopes = {
  scope:calls{
    to = "FunctionC",
    where = caller:named "FunctionA" and caller:has_call "FunctionB",
    using = {parameters = {var:named "VarA", _, _}},
    with = check
  }
}

function check(project, context) -- > Call Site Context
  local caller = context.caller.address
  local arg = context.inputs[1]

  if arg.annotation == "VarA" then
    print("Symbolic name propagated to call site")
  end

  -- ...
end
```

```
void FunctionA(void *VarA, int val, int size) {
  ...

  FunctionC(VarA);

  FunctionB(val, size)
}
```

Rule engine: Example / CVE-2022-23218

```
scopes = {
  scope:calls{
    to = "memcpy",
    where = caller:named "svcunix_create" and
      not caller:has_call "__sockaddr_un_set",
    using = {parameters = {_, _, _, var:named "Path"}},
    with = check
  }
}

function check(project, context)
  local src = context.inputs[2]

  if src ~= nil and src.annotation == "Path" then
    return result:critical{
      description = "Stack-based buffer overflow in the `sunrpc` module of the GNU C Library[...]",
      evidence = {
        functions = {
          [context.caller.address] = {
            annotate:prototype "SVCXPRT *svcunix_create (int sock, u_int sendsize, u_int recvsize, char *path)",
            annotate:at{
              location = context.caller.address,
              message = "The last argument of this function is attacker-controlled and its size is not validated"
            },
            annotate:at{
              location = context.caller.call_address,
              message = "It is then passed to this `memcpy` call, resulting in a stack-based buffer overflow"
            }
          }
        }
      }
    }
  end
end
```

```
SVCXPRT *
svcunix_create (int sock, u_int sendsize, u_int recvsize, char *path)
{
  ...

  len = strlen (path) + 1;
  memcpy (addr.sun_path, path, len);
}
```

CVE-2022-23218

Rule engine: Example / CVE-2022-23218

```
× Stack-based buffer overflow in the `sunrpc` module of the GNU C Library via the `path` parameter of the
| `svcunix_create` function
   ┌─[1:10]
 1 │ SVCXPRT *svcunix_create(int32_t sock, u_int sendsize, u_int recvsize, char *path) {
   ·          ──────────────
   ·                        └── The last argument of this function is attacker-controlled and its size is not validated
 2 │   bool var1;
 3 │   caddr_t var2;
 4 │   bits64_t var3;
 5 │   int32_t var4;
 6 │   uint64_t var5;
 7 │   char *var6;
 8 │   u_int *ptr;
 9 │   SVCXPRT *ptr_00;
10 │   bits64_t var7;
11 │   socklen_t stack_80 [4];
   ·
   ·
   ·
57 │   stack_40 = 0;
58 │   stack_38 = 0;
59 │   stack_30 = 0;
60 │   stack_28 = 0;
61 │   stack_20 = 0;
62 │   stack_18 = 0;
63 │   stack_10 = 0;
64 │   stack_8 = 0;
65 │   stack_4 = 0;
66 │   var5 = strlen(path);
67 │   memcpy(stack_70.sa_data, path, var5 + 1);
   ·   ──────────────────────────────────────────
   ·                          └── It is then passed to this `memcpy` call, resulting in a stack-based buffer overflow
68 │   stack_80[0] = var5 + 3;
```

Rule engine: Patch validation

- Validates whether a vulnerability has actually been patched in the binary, emitting a result:patch finding
- Reduces false positives compared to version-string detection

```
scopes = {scope:functions{named = "svcunix_create", with = check}}

function check(project, context)
  local calls = context:calls("__sockaddr_un_set")

  if #calls ~= 0 then
    return result:patch{
      evidence = {
        functions = {
          [context.address] = {
            annotate:prototype "SVCXPRT *svcunix_create (int sock, u_int sendsize, u_int recvsize, char *path)",
            annotate:at{
              location = calls[1],
              message = "This call to `__sockaddr_un_set` correctly validates the `path` parameter"
            }
          }
        }
      }
    }
  end
end
```

Demo: Identify real-world vulnerabilities

binarly

```
author = "Binarly"                                                    1/87
name = "CVE-2024-6387"
platform = "posix-binary"
architecture = "*:*:*"
conditions = {name_with_prefix = "sshd"}
types = "openssh/v8.7p1/openssh.bin"
signatures = {project = "sshd", from = "8.5p1", to = "9.7p1"}

scopes = {
  scope:calls{
    to = "sshlogv",
    where = caller:has_calls({"_exit"}),
    using = {},
    with = check
  }
}

local function is_alarm_handler(f)
  return f:has_call("getpgid") and f:has_call("getpid") and f:has_call("k
end

local function check(project, context)
  local sshlogv_location = context.caller.call_address
  local sshsigdie_location = context.caller.address

  local funcs = project:functions_where(is_alarm_handler)
  if #funcs == 0 then return end

  local alarm_handler = funcs[1]

  local sshsigdie_calls = alarm_handler:calls(sshsigdie_location)
  if #sshsigdie_calls == 0 then
    -- not vulnerable: `sshsigdie` is not reachable from the alarm handle
    return
  end

  -- at this point, we have proven that `sshsigdie` is reachable from the
  -- alarm handler and `sshlogv` is reachable from `sshsigdie`

  return result:high{
    name = "CVE-2024-6387",
    description = "Race condition in OpenSSH's sshd signal handling due t
    provenance = {
      kind = "posix.ELF",
      linkage = "project",
      vendor = "openssh",
      product = "sshd",
      license = "MIT",
      affected_versions = {"<4.4p1", ">=8.5", "<9.8"}
    },
```

```
tests/local/reverse_2026 on  platform-v3.0 [!?] via  v3.14.1 on ☁  francesco@bi
narly.io
❯
```

Introduction
VulHunt
Rule Engine

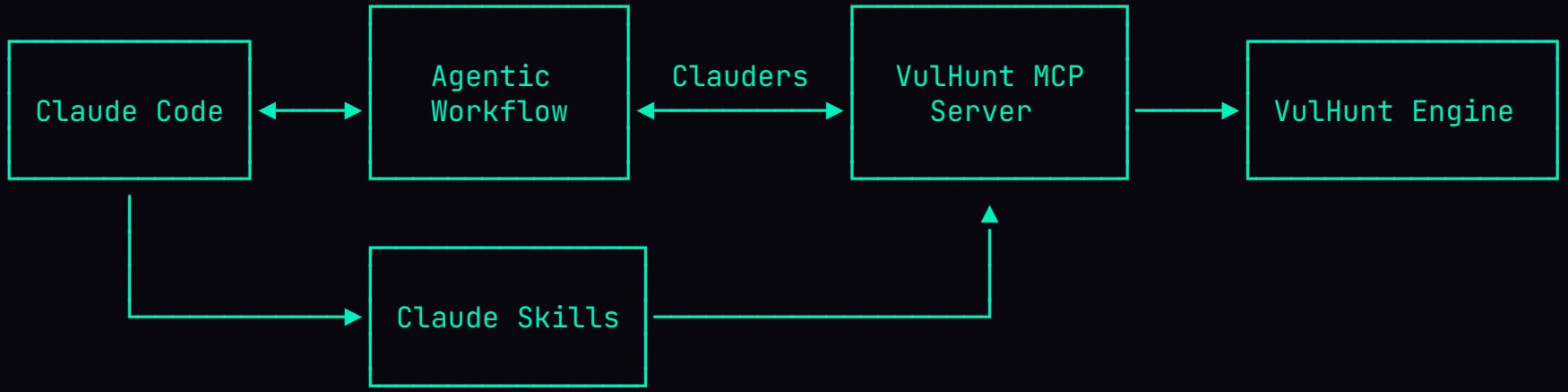
LLM Integration

Conclusion

binarly

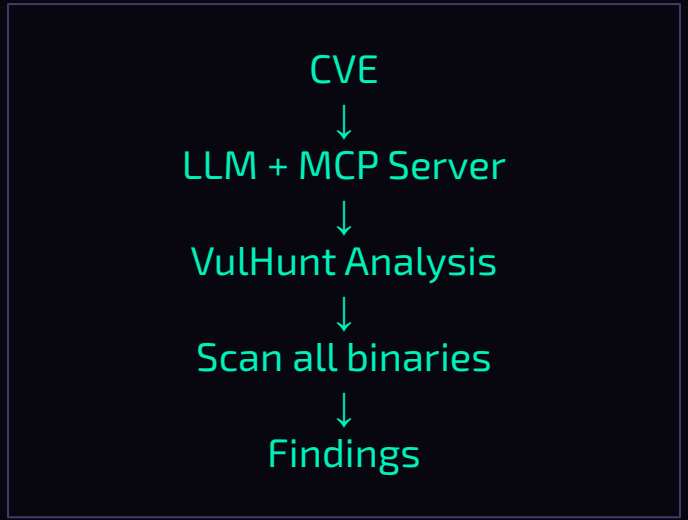
LLM integration: Architecture overview

- VulHunt APIs exposed via an MCP server
  - Load project, Run Lua queries
- Workflow using clauders: Rust bindings for the Claude Code CLI
- Pre-built Claude Skills for common tasks — leveraging VulHunt APIs via MCP to run targeted analyses



LLM integration: Vulnerability triaging

- Security teams receive thousands of CVE alerts
- Triaging requires reverse engineering expertise and is time-consuming
- Manual verification doesn't scale
- When new CVEs are disclosed, teams need immediate answers



Find and summarize details about CVE-2019-14889.
* Explain the vulnerability in clear, simple language.
* If possible, identify the specific vulnerable functions, the source of untrusted input, and the sink where the vulnerability is triggered.

Then, analyze the binary located at /tmp/libssh.so:
* Attempt to locate the vulnerable code paths or functions within the binary.
* Highlight where the vulnerability might exist and explain how it relates to the CVE details you found.

If the vulnerability is located in the binary:
* Decompile the vulnerable function(s).
* Add clear, concise annotations (no more than 5) describing:
  * The key vulnerable operations or function calls.
  * The impact of the vulnerability.
  * The type of vulnerability (e.g., buffer overflow, use-after-free).

Provide your reasoning step by step, and ensure the explanation is actionable and precise.

Show less

6:28 PM

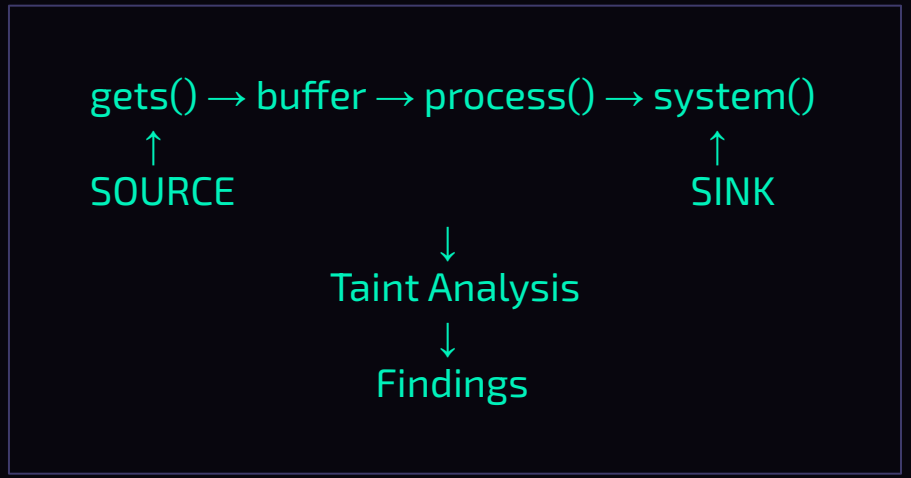
I'll start by researching CVE-2019-14889, then analyze the binary. Let me begin.

Reply...

Opus 4.6

LLM integration: Vulnerability hunting

- Context-dependent logic flaws are tough to identify
- LLMs can:
  - Reason about program context and how components interact
  - Dynamically search for common vulnerability patterns



```
❯ ./target/release/vulhunt-clauders --binary /tmp/rc
```

Introduction
VulHunt
Rule Engine
LLM Integration

Conclusion

binarly

What We're Releasing

- **VulHunt** — Vulnerability hunting framework
- **Binary analysis framework (BIAS)** — Rust-based foundation that VulHunt is built on
- **MCP server** — Exposes VulHunt's capabilities to LLM-powered tools and agents
- **Type library toolkit** — Utilities to build and inspect type libraries

All open source.

Conclusion

- Official website
- GitHub repository
- Slack workspace

Future Work

- Improved dataflow analysis
- …and much more on the roadmap

@vulhuntdev

VULHUNT.RE

CHAT WITH US

Rule engine: Modules

- Enables reusing primitives across rules

```
function x86.find_call(pattern)
  for _, insn in ipairs(pattern.insns) do
    if insn.mnemonic == "CALL" then return insn.address end
  end

  return nil
end
```

x86.vhm

```
function check(project)
  local x86 = require "uefi/primitives/x86"

  local code = project:search_code("488d40..bf........4c8bcb
  if code == nil then return end

  local call = x86.find_call(code)
end
```

rule.vh